

00_data_exploration

June 24, 2026

1 Step 0: Raw Data Exploration for the Authoritative v4 Line

This notebook documents the raw cohort and row-retention chain that feed the frozen v4 pre-processing line. It is intentionally descriptive rather than predictive: the purpose is to make the cohort auditable before any modeling claims are introduced. The two quantities to keep in view throughout Step 0 are the **63 total subjects** represented in the raw database and the **15,160 raw stride rows** that are progressively filtered down to the authoritative v4 analytical matrix.

```
[1]: import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import polars as pl

def find_repo_root() -> Path:
    candidates = [Path.cwd().resolve(), *Path.cwd().resolve().parents]
    for base in candidates:
        if (base / 'data' / 'processed' / 'v4' / 'v4_protocol_manifest.json').
↪exists():
            return base
    raise RuntimeError('Could not locate repository root from the current_
↪working directory.')

REPO_ROOT = find_repo_root()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
if str(REPO_ROOT / 'src') not in sys.path:
    sys.path.insert(0, str(REPO_ROOT / 'src'))

from src.preprocessing import load_raw_data, filter_pause_events,
↪filter_artifact_rows_v3

STEP_TAG = 'step0'
```

```

FIG_PDF = REPO_ROOT / 'report' / 'figures' / 'v4' / 'pdf' / STEP_TAG
FIG_PNG = REPO_ROOT / 'report' / 'figures' / 'v4' / 'png' / STEP_TAG
TABLE_DIR = REPO_ROOT / 'report' / 'tables' / 'v4'
for directory in (FIG_PDF, FIG_PNG, TABLE_DIR):
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    'font.family': 'DejaVu Serif',
    'font.size': 9,
    'axes.labelsize': 9,
    'axes.titlesize': 10,
    'xtick.labelsize': 8,
    'ytick.labelsize': 8,
    'legend.fontsize': 7,
    'figure.dpi': 180,
    'savefig.bbox': 'tight',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
})

IEEE_ONE_COL = 3.5
IEEE_TWO_COL = 7.16
COND_ORDER = ['pd', 'hd', 'als']
COND_LABEL = {'pd': 'PD', 'hd': 'HD', 'als': 'ALS', 'control': 'Control'}
COND_COLOR = {'pd': '#4C78A8', 'hd': '#F58518', 'als': '#54A24B', 'control': '
↳ '#7F7F7F'}

def savefig_stem(fig: plt.Figure, stem: str) -> None:
    pdf_path = FIG_PDF / f'{stem}.pdf'
    png_path = FIG_PNG / f'{stem}.png'
    fig.savefig(pdf_path)
    fig.savefig(png_path, dpi=300)
    print(f'Saved figure: {pdf_path.relative_to(REPO_ROOT)}')
    print(f'Saved figure: {png_path.relative_to(REPO_ROOT)}')

def save_table_csv(df: pd.DataFrame, name: str, *, index: bool = False) -> None:
    path = TABLE_DIR / name
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

RAW_DIR = REPO_ROOT / 'data' / 'raw' / '
↳ 'gait-in-neurodegenerative-disease-database-1.0.0'
protocol = json.loads((REPO_ROOT / 'data' / 'processed' / 'v4' / '
↳ 'v4_protocol_manifest.json').read_text())

```

```

preproc = json.loads((REPO_ROOT / 'data' / 'processed' / 'v4' /
↳ 'preprocessing_manifest_v4.json').read_text())

raw_df = load_raw_data(str(RAW_DIR))
pause_filtered = filter_pause_events(raw_df)
robust_filtered, robust_stats = filter_artifact_rows_v3(
    raw_df,
    robust_mad_multiplier=float(preproc['robust_mad_multiplier']),
    return_stats=True,
)

assert raw_df.height == preproc['raw_rows']
assert robust_filtered.height == preproc['final_rows']
assert robust_filtered['subject_id'].n_unique() == preproc['n_subjects']

print({
    'raw_rows': raw_df.height,
    'after_pause_filter_rows': pause_filtered.height,
    'final_rows_v4': robust_filtered.height,
    'n_subjects_final': robust_filtered['subject_id'].n_unique(),
})

```

```

{'raw_rows': 15160, 'after_pause_filter_rows': 14753, 'final_rows_v4': 13765,
'n_subjects_final': 63}

```

1.1 Raw cohort summary table

This table anchors the cohort description used throughout the rest of the project. The raw database contains **20 HD subjects**, **16 controls**, **15 PD subjects**, and **13 ALS subjects**; at the stride-row level, **HD contributes the largest raw block (4,846 rows)**, whereas **ALS is the smallest disease cohort (2,550 rows)**. That imbalance matters later because strong ALS performance in the supervised benchmarks still comes from a modest number of subjects.

```

[2]: subject_counts = (
    raw_df
    .group_by('condition')
    .agg(pl.col('subject_id').n_unique().alias('n_subjects'))
    .sort('condition')
)
raw_stride_counts = (
    raw_df
    .group_by('condition')
    .agg(pl.len().alias('raw_strides'))
    .sort('condition')
)
final_stride_counts = (
    robust_filtered
    .group_by('condition')

```

```

        .agg(pl.len().alias('final_strides'))
        .sort('condition')
    )

    cohort_table = (
        subject_counts
        .join(raw_stride_counts, on='condition', how='left')
        .join(final_stride_counts, on='condition', how='left')
        .to_pandas()
    )
    cohort_table['condition_label'] = cohort_table['condition'].map(COND_LABEL)

    filter_flow = pd.DataFrame([
        {'stage_key': 'raw', 'stage_label': 'raw', 'rows': raw_df.height},
        {
            'stage_key': 'after_pause_and_impossible_pct_filter',
            'stage_label': 'after pause +\nimpossible % filter',
            'rows': pause_filtered.height,
        },
        {
            'stage_key': 'after_authoritative_v4_robust_filter',
            'stage_label': 'after authoritative\nv4 robust filter',
            'rows': robust_filtered.height,
        },
    ])

    save_table_csv(cohort_table, 'step0_raw_cohort_summary.csv')
    save_table_csv(filter_flow, 'step0_filter_flow.csv')

    display(
        cohort_table.style.format({
            'n_subjects': '{:.0f}',
            'raw_strides': '{:,.0f}',
            'final_strides': '{:,.0f}',
        }).set_caption('Raw and final cohort counts aligned to the frozen v4_
↳ preprocessing manifest')
    )

```

Saved table: report/tables/v4/step0_raw_cohort_summary.csv

Saved table: report/tables/v4/step0_filter_flow.csv

<pandas.io.formats.style.Styler at 0x7f2eaa7f1820>

1.2 Raw cohort composition and authoritative filtering cascade

These two Step 0 figures serve complementary roles. The cohort-composition panel shows how raw subject counts and raw stride counts are distributed before any authoritative filtering, while the filtering cascade shows the exact retained-row path from **15,160** raw rows to **14,753** rows after

the pause/impossible-percentage filter and then to **13,765** rows after the authoritative v4 robust filter. Read together, they support a narrow but important claim: the final v4 matrix is traceable from the raw repository without any hidden exclusion stage.

```
[3]: fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.2, 2.8),
    ↪constrained_layout=True)
x = np.arange(len(cohort_table))
bars = ax.bar(
    x,
    cohort_table['n_subjects'],
    color=[COND_COLOR[c] for c in cohort_table['condition']],
    edgecolor='black',
    linewidth=0.6,
)
ax.set_xticks(x)
ax.set_xticklabels(cohort_table['condition_label'])
ax.set_ylabel('Subjects')
ax.set_title('Raw cohort composition')
ax.grid(axis='y', alpha=0.2)
ax.set_ylim(0, float(cohort_table['n_subjects'].max()) + 6)
for idx, bar in enumerate(bars):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.45,
        f"{int(cohort_table.loc[idx, 'raw_strides']):,} stride rows",
        ha='center',
        va='bottom',
        fontsize=8,
    )
savefig_stem(fig, 'step0_raw_cohort_composition')
plt.show()

fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 0.85, 3.3),
    ↪constrained_layout=True)
cascade_x = np.arange(len(filter_flow))
cascadeBars = ax.bar(
    cascade_x,
    filter_flow['rows'],
    color=['#D5D8DC', '#85C1E9', '#2E86C1'],
    edgecolor='black',
    linewidth=0.6,
    width=0.62,
)
ax.set_xticks(cascade_x)
ax.set_xticklabels(filter_flow['stage_label'])
ax.set_ylabel('Stride rows')
ax.set_title('Authoritative filtering cascade')
```

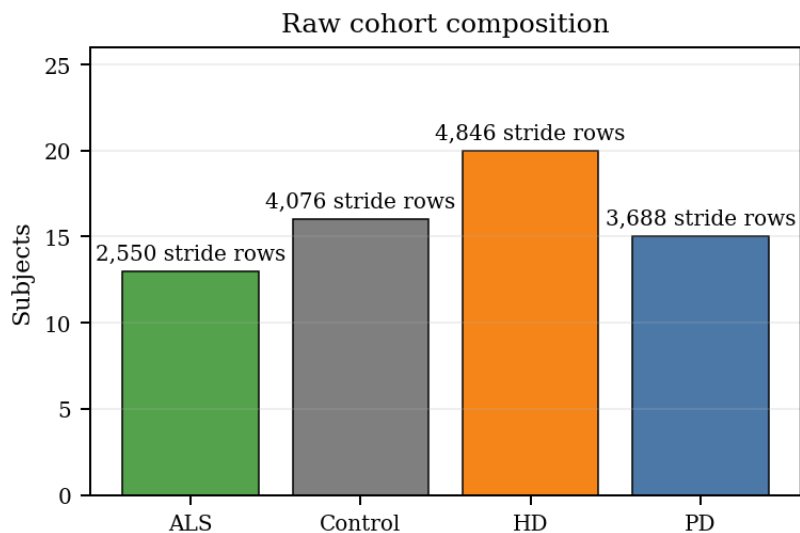
```

ax.grid(axis='y', alpha=0.2)
ymax = float(filter_flow['rows'].max())
ax.set_ylim(0, ymax * 1.18)
for idx, bar in enumerate(cascade_bars):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + ymax * 0.02,
        f"{int(filter_flow.loc[idx, 'rows']):,}",
        ha='center',
        va='bottom',
        fontsize=8,
    )
savefig_stem(fig, 'step0_authoritative_filtering_cascade')
plt.show()

```

Saved figure: report/figures/v4/pdf/step0/step0_raw_cohort_composition.pdf

Saved figure: report/figures/v4/png/step0/step0_raw_cohort_composition.png

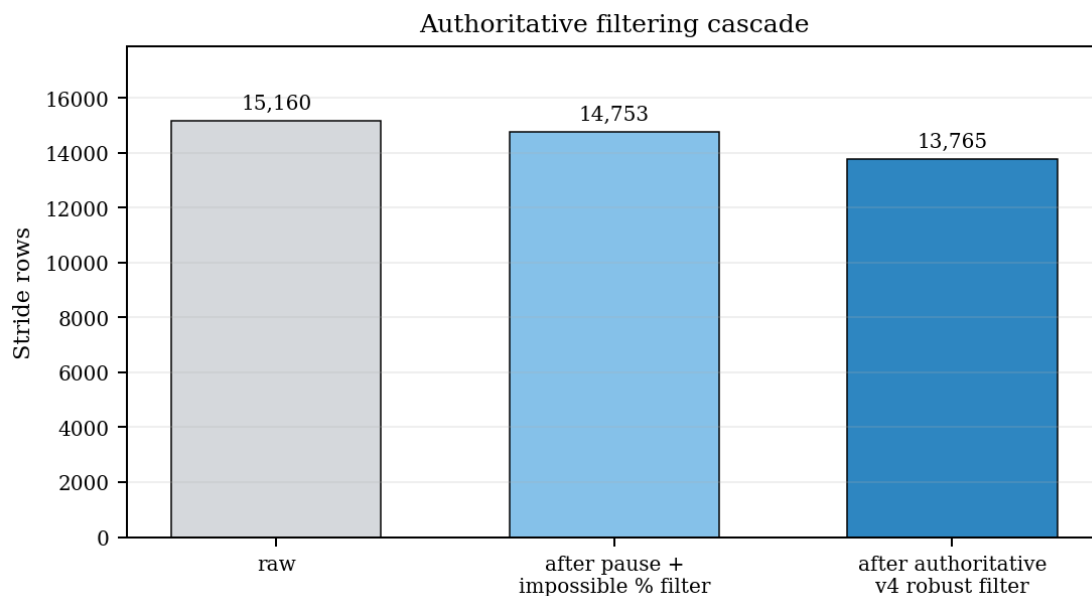


Saved figure:

report/figures/v4/pdf/step0/step0_authoritative_filtering_cascade.pdf

Saved figure:

report/figures/v4/png/step0/step0_authoritative_filtering_cascade.png



1.3 Interpretation notes

Step 0 should stay descriptive in the manuscript and repository alike. It supports cohort transparency, justifies later small-sample caveats, and shows that the retained-row losses are substantial enough to matter methodologically without suggesting a catastrophic preprocessing failure. The retained legacy symbol `filter_artifact_rows_v3` is implementation name only; the factual record in this notebook is the frozen **v4** filtering path and the saved Step 0 tables.